React onClick Listeners

In this lesson, we will be diving into adding event listeners to our application. These are the interactive elements that respond to user actions, such as clicks or keyboard inputs. Specifically, we will be focusing on the `onClick` event listener in this lecture.

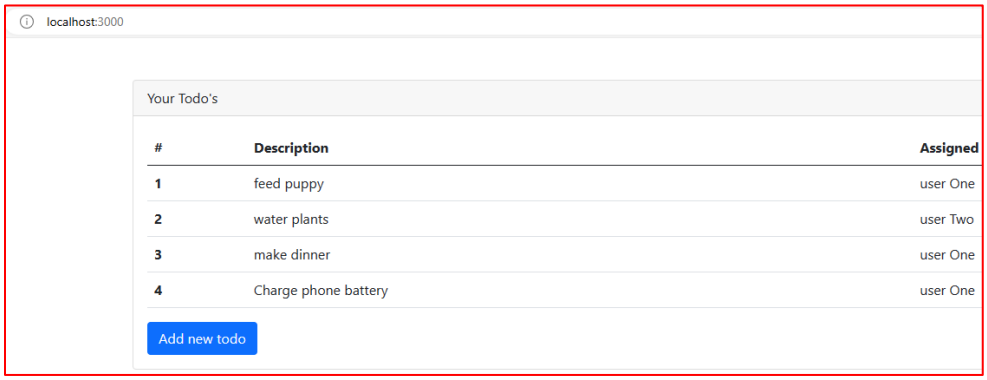
Currently, our application includes four to-do items. We've accomplished dynamic rendering of a to-do row item for each item in our `todos` list, but the functionality is still somewhat static. Let's add an interactive element.

Creating a Button: Right under our to-do table, where we are passing in `todos`, I'm going to create a button. Using JSX, I will create a button element and add Bootstrap class names, `button` and `button-primary`, to stylize it with a beautiful blue color. The button will be labeled "Add New To-Do".

```
<button className='btn btn-primary'>Add new todo</button>
```



When we jump back to the browser, we can see the "Add New To-Do" button in our to-do application. However, clicking on it currently has no effect. We'll change that next.

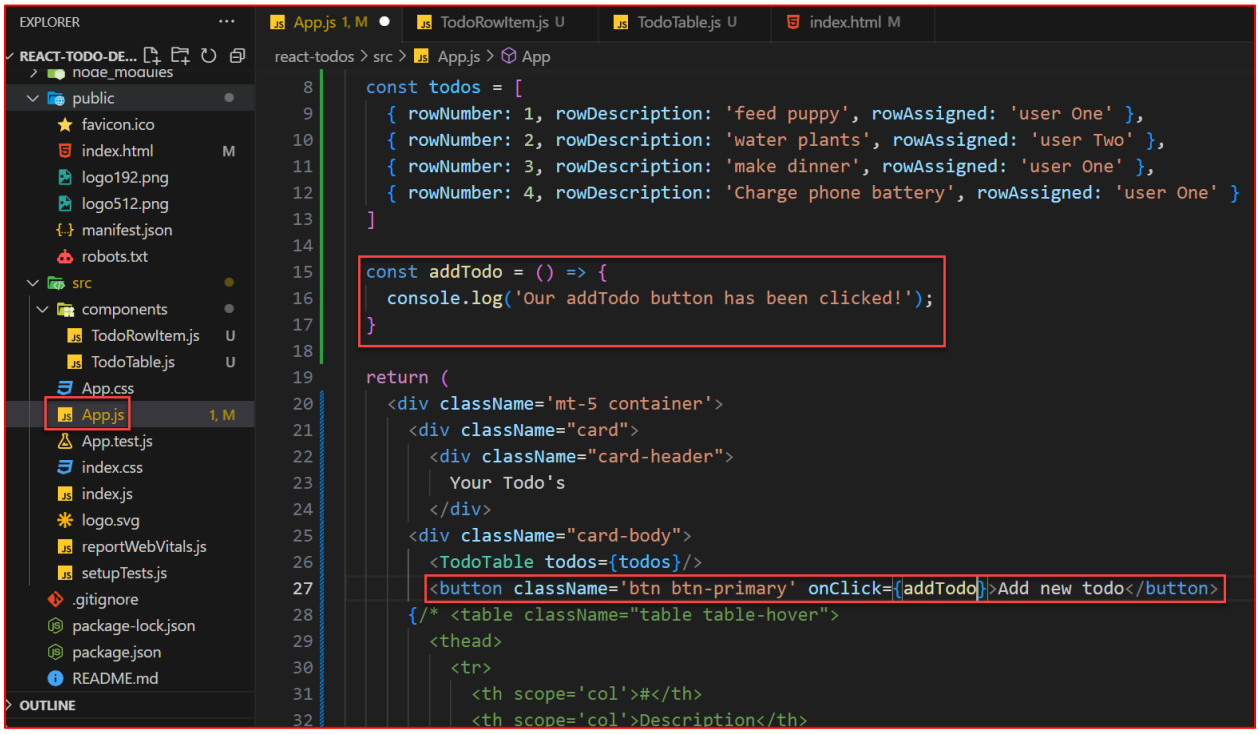


Adding an Event Listener: Let's go back into the code and add an `onClick` listener to the button. In React, the term `on` denotes that we're going to pass some type of event listener as a prop to our rendered JSX component. In this case, we want something to happen when the button is clicked, so we'll use `onClick`.

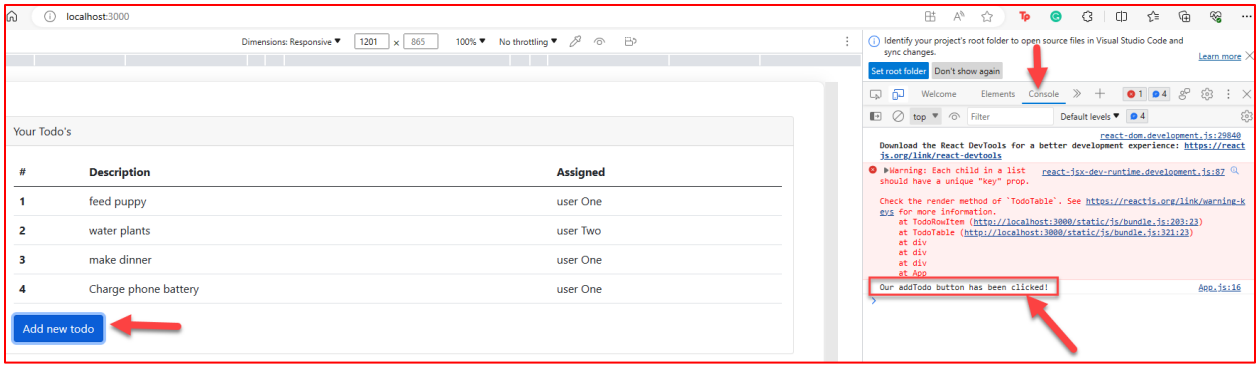
We can either use a function that we create above the return statement or an arrow function using `const`. Here's what we are going to do:

```
const addToDo = () => {
  console.log('Our Add New To-Do button was clicked.');
};
```

After defining this arrow function, we'll go back to our `onClick` and type in `addToDo`. Important note: Make sure you do not add parentheses at the end. We want to reference the function, not call it right away.

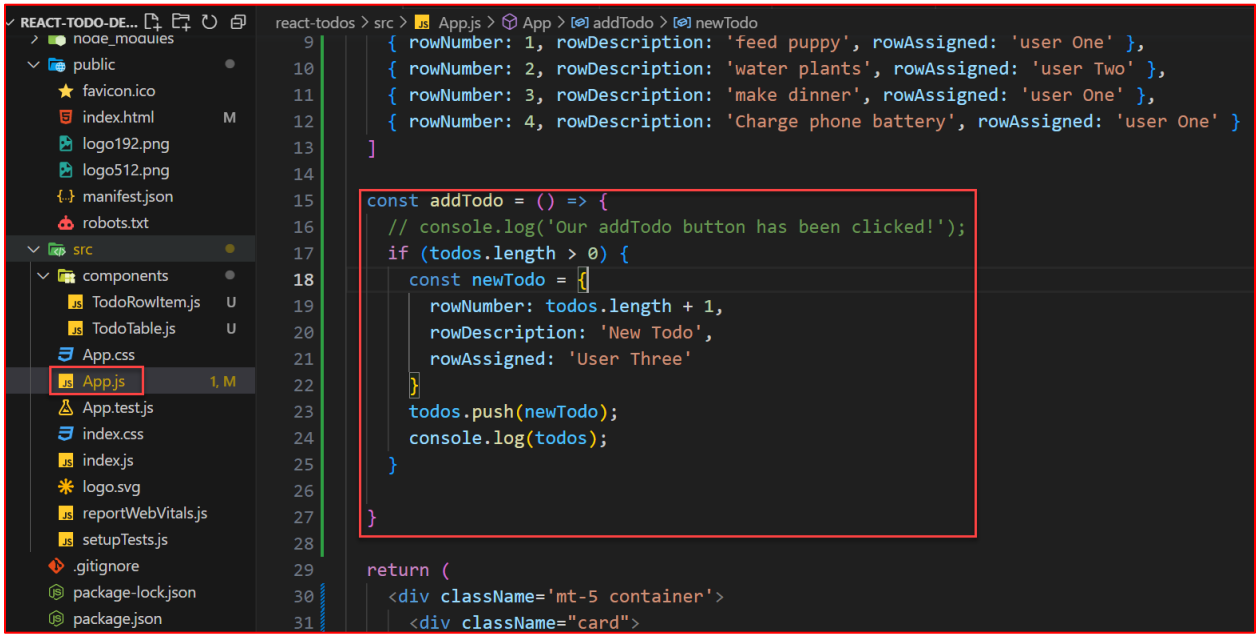


If we now go back to our browser, refresh the page, and click the "Add New To-Do" button, we can see in the console that it prints the message we defined. We're effectively telling React that, when the button is clicked, we want to call the function that this `onClick` is pointing to.

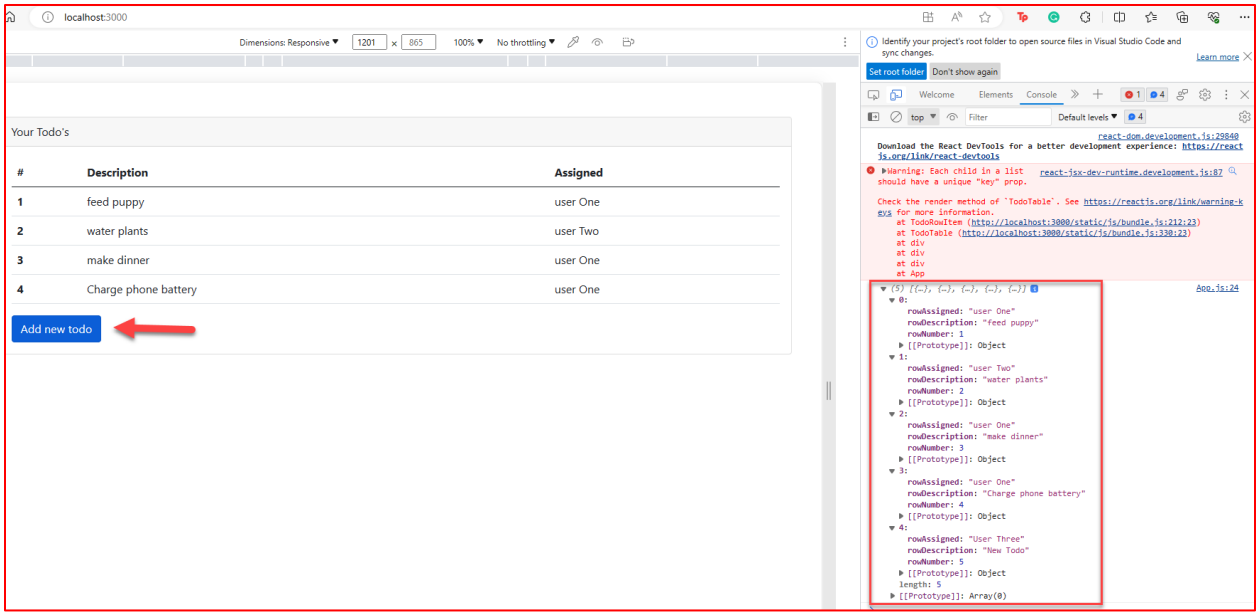


Why No Parentheses?: If we had added parentheses at the end, JavaScript would have executed the `addToDo` function automatically when the button gets rendered, which is not what we want. We want the function to be triggered only when it's clicked. Thus, we ensure it's a pointer by omitting the parentheses.

Adding More Functionality: Now, let's enhance our `addTodo` function. We'll ensure that there's at least one element in the `todos` array and then create a new to-do object with a unique row number and predefined description and assigned user. After pushing this new object to our `todos` array, we'll log all the to-dos to the console.



By clicking the "Add New Todo" button repeatedly, we can observe that new elements are added to the todo list array. This process illustrates how we can manipulate arrays through user interactions.



Handling State: While our console reflects the additions, they are not yet visible in our rendered application. We need to manage the state of this React application to enable automatic re-rendering and immediate visualization of the changes. We'll cover how to do this in the upcoming lessons.

Alternative Approaches: We've looked at one way to create an event listener using a named arrow function, but we could also use an anonymous function or even define a function within the `onClick` listener itself. However, for the sake of code cleanliness, it's best to keep our JSX minimal and use pointers to named functions.

Summary: Through this lesson, we've learned how to add an `onClick` listener to a button in React, create and call a function in response to a click event, and perform dynamic actions like adding new items to an array. In the next lesson, we will further explore handling state to make our application more responsive to user interactions.